

Pandas统计分析基础



● 学习目标



- ◆ 掌握pandas读写数据
- ◆ 掌握基本的数据清洗方法
- ◆ 掌握基本的数据整理方法

Pandas中常用的读写不同格式文件的函数如表所示。

读取函数	写入函数	描述
<code>read_csv()</code>	<code>to_csv()</code>	读写csv格式的数据
<code>read_table()</code>		读取普通分隔符分割的数据
<code>read_excel()</code>	<code>to_excel()</code>	读写excel格式的数据
<code>read_json()</code>	<code>to_json()</code>	读写json格式的数据
<code>read_html()</code>	<code>to_html()</code>	读写html格式的数据
<code>read_sql()</code>	<code>to_sql()</code>	读写数据库中的数据

❖ 读写csv文件

(1) 读取csv文件中的数据

student.csv文件内容如下：

Name, Math, Physics, Chemistry

WangLi, 93, 88, 90

ZhangHua, 97, 86, 92

LiMing, 84, 72, 77

ZhouBin, 97, 94, 80

这个文件以逗号作为分隔符，可使用pandas的read_csv()函数读取它的内容，返回DataFrame格式的文件。

❖ 读写csv文件

(1) 读取csv文件中的数据

```
In [2]: ▶ import pandas as pd  
#从csv中读取数据  
csvframe = pd.read_csv('student.csv')  
print(csvframe)
```

	Name	Math	Physics	Chemistry
0	WangLi	93	88	90
1	ZhangHua	97	86	92
2	LiMing	84	72	77
3	ZhouBin	97	94	80

❖ 读写csv文件

(1) 读取csv文件中的数据

csv文件中的数据为列表数据，位于不同列的元素用逗号隔开，csv文件被视作文本文件，也可以使用pandas的read_table()函数读取，但需要指定分隔符。

```
In [3]: ▶ pd.read_table('student.csv', sep=',',  
print(csvframe))
```

	Name	Math	Physics	Chemistry
0	WangLi	93	88	90
1	ZhangHua	97	86	92
2	LiMing	84	72	77
3	ZhouBin	97	94	80

❖ 读写csv文件

(1) 读写csv文件

pd.read_csv(filepath_or_buffer, sep=',', header='infer', names=None, index_col=None, usecols=None)

作用：读取csv（逗号分割）文件到DataFrame对象。

filepath_or_buffer：拟要读取的文件的路径。

sep：其类型是str，默认‘,’，用来指定分隔符，csv文件中的分隔符一般为逗号分隔符。

header：指定第几行作为列名，默认为0（第一行）。如果第一行不是列名，是内容，可以设置header=None，以便不把第一行当作列名。

names：用于结果的列名列表，对各列重命名，即添加表头。

index_col：用作行索引的列编号或者列名，可使用index_col=[0, 1]来指定文件中的第1和2列为行索引。

usecols：选取某几列，不读取整个文件的内容，如usecols=[1, 2]。

❖ 读写csv文件

(1) 读写csv文件

header参数可以是一个列表，例如[0, 2]，这个列表表示将文件中的这些行作为列标题，介于中间的行将被忽略掉（例如下例中的行号为0、2的行将被作为多级标题出现，行号为1的行将被丢弃，**dataframe**的数据从行号为3的行开始）。

❖ 读写csv文件

(1) 读写csv文件

```
▶ print(csvframe)
#指定csv文件中的行号为0、2的行为列标题
csvframe = pd.read_csv('student.csv', header=[0, 2])
print(csvframe)
```

	Name	Math	Physics	Chemistry
0	WangLi	93	88	90
1	ZhangHua	97	86	92
2	LiMing	84	72	77
3	ZhouBin	97	94	80

	Name	Math	Physics	Chemistry
	ZhangHua	97	86	92
0	LiMing	84	72	77
1	ZhouBin	97	94	80

❖ 读写csv文件

(2) 往csv文件写入数据

把DataFrame对象中的数据写入csv文件，要用到to_csv()函数：

```
DataFrame.to_csv(path_or_buf=None, sep=',', na_rep='',  
columns=None, header=True, index=True)
```

作用：以逗号为分隔将DataFrame对象中的数据写入csv文件中。

filepath_or_buffer：拟要写入的文件的路径或对象。

sep：默认字符','，用来指定输出文件的字段分隔符。

na_rep：字符串，默认''，缺失数据表示。

columns：指定要写入文件的列。

header：是否保存列名，默认为True，保存。如果给定字符串列表，则将其作为列名的别名。

index：是否保存行索引，默认为True，保存。

❖ 读写csv文件

(2) 往csv文件写入数据

```
► import csv

df10=pd.DataFrame([[12, 3, 5], [13, 8, 7], [15, 13, 12], [22, 18, 15]],
                   index=['2018-08-01', '2018-08-02', '2018-08-03', '2018-08-04'],
                   columns=['book', 'box', 'pen'])

print(df10)
df10.to_csv('bbp1.csv', index=False, header=False)
with open('bbp1.csv', newline='') as csvfile:
    linereader=csv.reader(csvfile)
    for row in linereader:
        print(','.join(row))
```

	book	box	pen
2018-08-01	12	3	5
2018-08-02	13	8	7
2018-08-03	15	13	12
2018-08-04	22	18	15
12, 3, 5			
13, 8, 7			
15, 13, 12			
22, 18, 15			

❖ 读取text文件

pandas的函数read_table()可读取txt文本文件。

pandas.read_table(filepath_or_buffer, sep='\t', header='infer', names=None, index_col=None, skiprows=None, nrows=None, delim_whitespace=False)

作用：读取以'\t'分割的文件，返回DataFrame对象。

参数说明：

sep：其类型是str，用来指定分隔符，默认为制表符，可以是正则表达式。

index_col：指定行索引。

skiprows：用来指定读取时要排除的行。

nrows：从文件中要读取的行数。

delim_whitespace：delim_whitespace=True表示用空格来分隔每行。

❖ 读取text文件

首先在工作目录下创建名为11.txt的文本文件，其内容如下：

C Python Java

1 4 5

3 3 4

4 2 3

2 1 1

```
In [10]: ▶ print(pd.read_table('11.txt')) #读取11.txt文本文件
```

	C	Python	Java
0	1	4	5
1	3	3	4
2	4	2	3
3	2	1	1

❖ 读写Excel文件

(1) 读Excel文件

pandas提供了`read_excel()`函数来读取Excel文件，用`to_excel()`函数往Excel文件写入数据。

`pandas.read_excel(io, sheet_name=0, header=0, names=None, index_col=None, usecols=None, skiprows=None, skip_footer=0)`

作用：读取Excel文件中的数据，返回一个DataFrame对象。

参数说明：

io：Excel文件路径，是一个字符串。

sheet_name：返回指定的sheet（表），如果将`sheet_name`指定为`None`，则返回全表；如果需要返回多个表，可以将`sheet_name`指定为一个列表，例如`['sheet1', 'sheet2']`；可以根据sheet的名字字符串或索引来指定所要选取的sheet，例如`[0, 1, 'SEET5']`将返回第一、第二和第五个表；默认返回第一个表。

❖ 读写Excel文件

(1) 读Excel文件

pandas.read_excel(io, sheet_name=0, header=0, names=None, index_col=None, usecols=None, skiprows=None, skip_footer=0)

作用：读取Excel文件中的数据，返回一个DataFrame对象。

参数说明：

header：指定作为列名的行，默认0，即取第一行，数据为列名行以下的数据；若数据不含列名，则设定**header = None**。

names：指定所生成的DataFrame对象的列的名字，传入一个list数据。

index_col：指定某列为行索引。

usecols：通过名字或索引值读取指定的列。

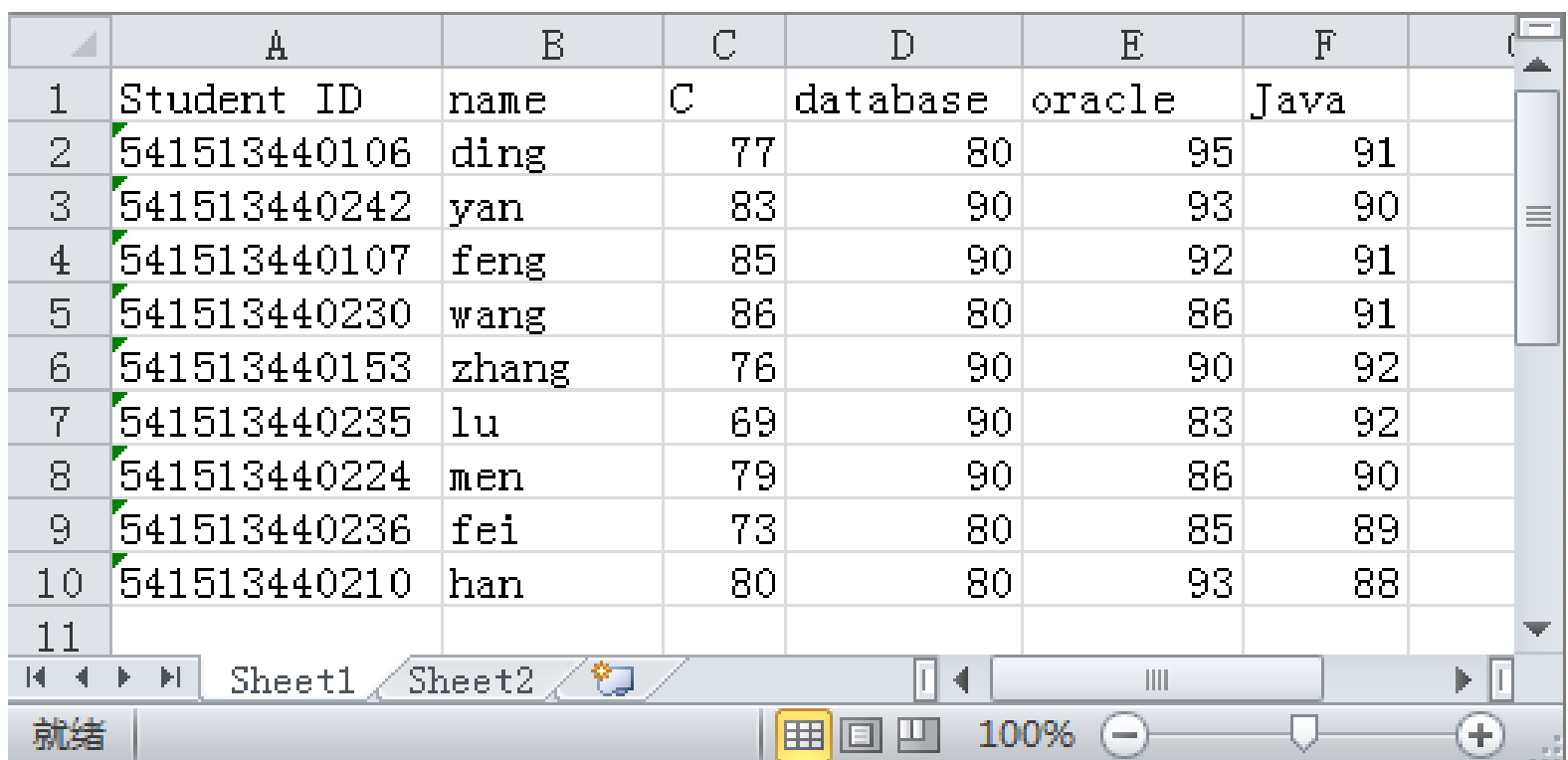
skiprows：省略指定行数的数据。

skip_footer：int，默认值为0，读取数据时省略最后int行。

❖ 读写Excel文件

(1) 读Excel文件

首先在工作目录下创建名为chengji.xlsx的Excel文件，Sheet1的内容表所示。



The image shows a screenshot of an Excel spreadsheet with the following data:

	A	B	C	D	E	F
1	Student ID	name	C	database	oracle	Java
2	541513440106	ding	77	80	95	91
3	541513440242	yan	83	90	93	90
4	541513440107	feng	85	90	92	91
5	541513440230	wang	86	80	86	91
6	541513440153	zhang	76	90	90	92
7	541513440235	lu	69	90	83	92
8	541513440224	men	79	90	86	90
9	541513440236	fei	73	80	85	89
10	541513440210	han	80	80	93	88
11						

The spreadsheet interface includes a status bar at the bottom with the text "就绪" (Ready) and a zoom level of 100%.

❖ 读写Excel文件

(1) 读Excel文件

▶ #将chengji.xlsx的列名作为所生成的DataFrame对象的第一行数据，并重新生成索引
`print(pd.read_excel('chengji.xlsx', header=None))`

	0	1	2	3	4	5
0	Student ID	name	C	database	oracle	Java
1	541513440106	ding	77	80	95	91
2	541513440242	yan	83	90	93	90
3	541513440107	feng	85	90	92	91
4	541513440230	wang	86	80	86	91
5	541513440153	zhang	76	90	90	92
6	541513440235	lu	69	90	83	92
7	541513440224	men	79	90	86	90
8	541513440236	fei	73	80	85	89
9	541513440210	han	80	80	93	88

❖ 读写Excel文件

(2) 往Excel文件写入数据

把DataFrame对象df中的数据写入Excel文件的函数为df.to_excel()。

df.to_excel()(excel_writer, sheet_name='Sheet1', na_rep='', columns=None, header=True, index=True, index_label = None, startrow=0, startcol=0, engine=None)

excel_writer: 输出路径。

sheet_name: 将数据存储在Excel的哪个Sheet页面，如Sheet1页面。

na_rep: 缺失值填充。

columns: 选择输出的的列。

header: 指定列名，布尔或字符串列表，默认为True，如果给定字符串列表，则假定它是列名称的别名。**header = False**则不输出题头。

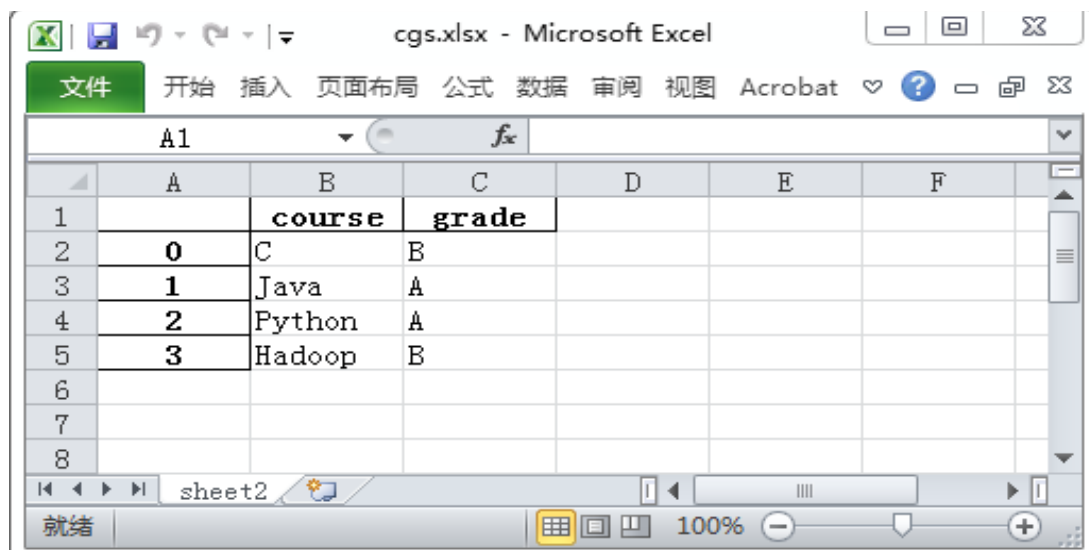
index: 布尔型，默认True，显示行索引（名字），当**index=False**，则不显示行索引（名字）。

❖ 读写Excel文件

(2) 往Excel文件写入数据

`sheet_name="sheet2"` 表示将 `df` 存储在 Excel 的 `sheet2` 页面，`columns=["course","grade"]` 表示选择“course”、“grade”两列进行输出，生成的 `cgs.xlsx` 文件表中的内容如下表所示。

```
df11=pd.DataFrame([[ 'C', 'B', 32], [ 'Java', 'A', 96],  
                    [ 'Python', 'A', 92], [ 'Hadoop', 'B', 88]],  
                    columns=[ 'course', 'grade', 'scores'])  
df11.to_excel(excel_writer='cgs.xlsx', sheet_name="sheet2", columns=["course", "grade"])
```



	A	B	C	D	E	F
1		course	grade			
2	0	C	B			
3	1	Java	A			
4	2	Python	A			
5	3	Hadoop	B			
6						
7						
8						

❖ 数据预处理的重要性

- 现实中采集到的原始数据往往不够规范，会存在一定程度的**缺失值**、**重复值**以及**噪声数据**等。直接采用这些数据进行分析有很大的可能会得到错误的结论。因此，在开始数据处理之前，需要**对数据进行预处理**。
- 垃圾数据进→垃圾结论出

❖ 数据预处理

DataFrame对象的数据预处理方法	描述
<code>df.duplicated(subset=None, keep='first')</code>	针对某些列，返回用布尔序列表示的重复行
<code>df.drop_duplicates(subset=None, keep='first', inplace=False)</code>	<code>df.drop_duplicates()</code> 用于删除df中重复行，并返回删除重复后的结果
<code>df.fillna(value=None, method=None, axis=None, inplace=False, limit=None)</code>	使用指定的方法填充NA/NaN缺失值
<code>df.drop(labels=None, axis=0, index=None, columns=None, inplace=False)</code>	删除指定轴上的行或列，它不改变原有的DataFrame对象中的数据，而是返回另一个DataFrame对象来存放删除后的数据

❖ 数据预处理

DataFrame对象的数据预处理方法	描述
<code>df.dropna(axis=0,how='any',thresh=None,subset=None,inplace=False)</code>	删除指定轴上的缺失值
<code>del df['col']</code>	直接在df对象上删除col列
<code>df.columns = col_lst</code>	重新命名列名，col_lst为自定义列名列表
<code>df.rename(index={'row1':'A'},columns= {'col1':'A1'})</code>	重命名行索引名和列索引名。
<code>df.reindex(index=None,columns=None,fill_value='NaN')</code>	改变索引，返回一个重新索引的新对象，index用作新行索引，columns用作新列索引，将缺失值填充为fill_value

❖ 数据预处理

DataFrame对象的数据预处理方法	描述
<code>df.replace(to_replace=None, value=None, inplace=False, limit=None, regex=False, method='pad')</code>	用来把to_replace所列出的且在df对象中出现的元素值替换为value所表示的值
<code>df.merge(right, how='inner', on=None, left_on=None, right_on=None)</code>	通过行索引或列索引进行两个DataFrame对象的连接
<code>pandas.concat(objs, axis=0, join='outer', join_axes=None, ignore_index=False, keys=None)</code>	以指定的轴将多个对象堆叠到一起, concat()不会去重对象中重复的记录
<code>df.stack(level=-1, dropna=True)</code>	将df的列旋转成行
<code>df.unstack(level=-1, fill_value=None)</code>	将df的行旋转为列

❖ 数据预处理的流程

- 数据预处理一般分三个步骤：**数据清洗**、**数据整理**以及**数据写入**。
- 在数据分析中，**数据清洗**是数据价值链中**最关键的**步骤。
- 数据清洗就是**处理缺失数据**以及**清除无意义**的信息。对于**垃圾数据**，即使是通过最好的分析，也将产生**错误的结果**，并**误导业务**本身。

❖ 数据清洗

- 原始数据中因为各种原因，可能存在缺失和错误。一般表现为数据的某些特征缺失，有噪声和不一致。**数据清洗过程首先识别出缺失的值，噪声数据和异常数据点等，并纠正数据中的缺失和错误。**
- 数据清洗通常包括4个步骤：
 - 1) 处理重复值
 - 2) 处理缺失值
 - 3) 处理异常值
 - 4) 处理数据格式错误

❖ 查看数据（部分数据）

- 查看头部数据，使用**df.head()**会默认打印出前5行。

df.head() # 查看前5行数据，df为DataFrame对象

df.head(2) # 查看数据前2行数据，括号里指定行数，就会打印指定的前几行数据。

- 查看尾部数据，使用**df.tail()**会默认打印出后5行。

df.tail() # 查看后5行数据，df为DataFrame对象

df.tail(n) # n为数字几，就是查看尾部几行的数据。



❖ 查看数据（部分数据）

```
# 导入 pandas 库并将其简化为 pd
import pandas as pd
# 读取口罩厂数据，文件路径为根目录
mask_data = pd.read_csv('mask_data.csv')
# 查看 mask_data
print(mask_data.head())
print(mask_data.tail())
```

	订单编号	日期	省	订单量	单价	销售额
0	87af-48e5-8bed-c5dcf9ecc172	2020-01-01	广东	0.0	30.0	0.0
1	535a-4eca-8fa0-9cc54c66e11d	2020-01-01	河南	0.0	30.0	0.0
2	a56d-4415-ad6e-020cdb154c35	2020-01-01	湖北	1.0	30.0	30.0
3	535a-4eca-8fa0-9cc54c66e11d	2020-01-01	河南	0.0	30.0	0.0
4	535a-4eca-8fa0-9cc54c66e11d	2020-01-01	河南	0.0	30.0	0.0

	订单编号	日期	省	订单量	单价 \
101937	6339-43f4-a12b-7b57d1ce3a48	NaN	NaN	NaN	NaN
101938	5e43-41c2-8696-fd5909cc98a4	NaN	NaN	999999999.0	999999999.0
101939	824c-4f2b-af43-038938d2db46	NaN	NaN	999999999.0	999999999.0
101940	1cf0-483a-ab11-0cebc6406c78	NaN	NaN	0.0	NaN
101941	8318-43df-9d34-0656d9eb759a	NaN	NaN	0.0	NaN

	销售额
101937	NaN
101938	999999999.0
101939	999999999.0
101940	0.0
101941	0.0



❖ 查看数据（全部数据）

```
# 导入 pandas 库并将其简化为 pd
import pandas as pd
# 读取口罩厂数据，文件路径为根目录
mask_data = pd.read_csv('mask_data.csv')
# 查看 mask_data
mask_data
```

	订单编号	日期	省	订单量	单价	销售额
0	87af-48e5-8bed-c5dcf9ecc172	2020-01-01	广东	0.0	30.0	0.0
1	535a-4eca-8fa0-9cc54c66e11d	2020-01-01	河南	0.0	30.0	0.0
2	a56d-4415-ad6e-020cdb154c35	2020-01-01	湖北	1.0	30.0	30.0
3	535a-4eca-8fa0-9cc54c66e11d	2020-01-01	河南	0.0	30.0	0.0
4	535a-4eca-8fa0-9cc54c66e11d	2020-01-01	河南	0.0	30.0	0.0
...	...	...	...	...	...	...
101937	6339-43f4-a12b-7b57d1ce3a48	NaN	NaN	NaN	NaN	NaN
101938	5e43-41c2-8696-fd5909cc98a4	NaN	NaN	999999999.0	999999999.0	999999999.0
101939	824c-4f2b-af43-038938d2db46	NaN	NaN	999999999.0	999999999.0	999999999.0
101940	1cf0-483a-ab11-0cebc6406c78	NaN	NaN	0.0	NaN	0.0
101941	8318-43df-9d34-0656d9eb759a	NaN	NaN	0.0	NaN	0.0

101942 rows × 6 columns

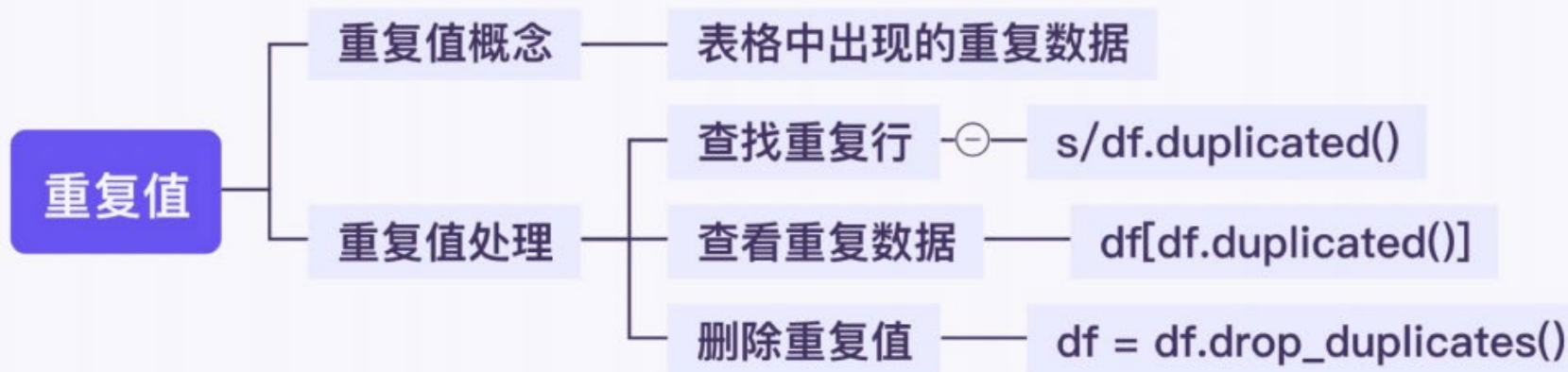
❖ 查看数据（全部数据）

这份数据存在一些问题，下面我们来解决这些问题。

	订单编号	日期	省	订单量	单价	销售额
0	87af-48e5-8bed-c5dcf9ecc172	2020-01-01	广东	0.0	30.0	0.0
1	535a-4eca-8fa0-9cc54c66e11d	2020-01-01	河南	0.0	30.0	0.0
2	a56d-4415-ad6e-020cdb154c35	2020-01-01	湖北	1.0	30.0	30.0
3	535a-4eca-8fa0-9cc54c66e11d	2020-01-01	河南	0.0	30.0	0.0
4	535a-4eca-8fa0-9cc54c66e11d	2020-01-01	河南	0.0	30.0	0.0
...	...	...	...	...	...	...
101937	6339-43f4-a12b-7b57d1ce3a48	NaN	NaN	NaN	NaN	NaN
101938	5e43-41c2-8696-fd5909cc98a4	NaN	NaN	999999999.0	999999999.0	999999999.0
101939	824c-4f2b-af43-038938d2db46	NaN	NaN	999999999.0	999999999.0	999999999.0
101940	1cf0-483a-ab11-0cebc6406c78	NaN	NaN	0.0	NaN	0.0
101941	8318-43df-9d34-0656d9eb759a	NaN	NaN	0.0	NaN	0.0

❖ 重复值处理

- 重复值很好理解，就跟字面意思一样，指的是表格中重复出现的数据。
- **处理重复值**的方法非常简单，发现重复值，**删除**即可，这样可以避免重复处理一些不完整的数据。



❖ 重复值处理

● 查看重复值: `data.duplicated(subset, keep)`

`subset` 参数可以传入具体的列名或列名序列，来精准定位查看某列的重复值，默认为None。

`keep` 参数可以传入 `{'first', 'last', False}`，默认为 `'first'`，只显示后面出现的重复值。如果要一起显示重复与被重复的数据，可以将`keep`参数的值设置为 `False`。

`data.duplicated(subset, keep)` 返回的是一个存有布尔值的 **Series 对象**，如果想查看具体的值，可以直接布尔索引 `data[data.duplicated(subset, keep)]`。

❖ 重复值处理

● 查看重复值

```
data={"name":["Alice","Jack","John","Mary","Alice",  
            "Jack","John","Mary"],  
      "age":[40,10,30,20,40,10,30,20],  
      "weight":[65,30,50,40,65,30,50,40]}  
df=pd.DataFrame(data)  
#显示布尔值  
print(df.duplicated())  
#当keep为False时, 显示所有重复值  
print(df[df.duplicated(keep=False)])  
#keep值默认为first, 保留第一次出现的值, 只显示后面出现的重复值  
print(df[df.duplicated()])
```

```
0    False  
1    False  
2    False  
3    False  
4     True  
5     True  
6     True  
7     True
```

dtype: bool

	name	age	weight
0	Alice	40	65
1	Jack	10	30
2	John	30	50
3	Mary	20	40
4	Alice	40	65
5	Jack	10	30
6	John	30	50
7	Mary	20	40

	name	age	weight
4	Alice	40	65
5	Jack	10	30
6	John	30	50
7	Mary	20	40

❖ 重复值处理

● **删除重复值**：`data.drop_duplicates(subset, keep, inplace)`

`subset` 参数可以传入具体的列名或列名序列，来精准定位删除某列的重复值，默认为`None`。

`keep` 参数可以传入 `{'first', 'last', False}`，默认为 `'first'`，保留第一次出现的重复值，删除后面的重复值。

`inplace` 参数可以传入 `True` 或 `False`，默认为 `False`，`True` 表示直接在原来的`DataFrame`上删除重复项，而默认值`False`表示生成一个副本。

❖ 重复值处理

● 删除重复值

```
data={"name":["Alice","Jack","John","Mary","Alice",  
            "Jack","John","Mary"],  
      "age":[40,10,30,20,40,10,30,20],  
      "weight":[65,30,50,40,65,30,50,40]}  
df=pd.DataFrame(data)  
print(df)  
#查看重复数据  
print(df[df.duplicated()])  
#去除数据重复值  
df.drop_duplicates(inplace=True)  
#验证, 重新查看是否有重复值  
print(df[df.duplicated(keep=False)])  
print(df)
```

	name	age	weight
0	Alice	40	65
1	Jack	10	30
2	John	30	50
3	Mary	20	40
4	Alice	40	65
5	Jack	10	30
6	John	30	50
7	Mary	20	40

	name	age	weight
4	Alice	40	65
5	Jack	10	30
6	John	30	50
7	Mary	20	40

Empty DataFrame
Columns: [name, age, weight]
Index: []

	name	age	weight
0	Alice	40	65
1	Jack	10	30
2	John	30	50
3	Mary	20	40

❖ 缺失值处理

● **数据缺失**，是经常遇到的情况，有一部分是源于现实数据不好获得，一部分则是源于人为因素。遇到数据缺失的情况，一般有两种方法进行处理：**直接删除**和**数据填补**。

● 直接删除数据是最简便的方法。但这建立在数据量够多的情况下。此时，删除几个空缺值，对整个数据不会构成较大影响。

❖ 缺失值处理

● 缺失值检查：

info() 查看数据信息

在实际的数据分析项目中，一般使用**info()**方法查看数据的缺失情况。当**非空数据**与**数据总量不一致**时，就说明这份数据有可能**存在缺失值**。

❖ 缺失值处理

● 缺失值检查:

```
# 导入 pandas 库并将其简化为 pd
import pandas as pd
# 读取口罩厂数据，文件路径为根目录
mask_data = pd.read_csv('mask_data.csv')
# 查看 mask_data 数据的基本信息
mask_data.info()
```

● 数据预处理



❖ 缺失值处理

● 缺失值检查:

总行数

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 101942 entries, 0 to 101941  
Data columns (total 6 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   订单编号    101942 non-null object  
1   日期        100956 non-null object  
2   省          100956 non-null object  
3   订单量      100960 non-null float64  
4   单价        100958 non-null float64  
5   销售额      100960 non-null float64  
dtypes: float64(3), object(3)  
memory usage: 4.7+ MB
```

各列总共有多少非空数据

各列的列名

各列数据类型统计

表格占用的系统空间

❖ 缺失值处理

● 查看缺失值：

data.isnull() 查找缺失值

data.isnull() 返回的是 **DataFrame** 类型的布尔值数据，检测结果为布尔值：缺失数据会用 **True** 来表示，**False** 则代表这里的数据正常。**True** 的位置标识该位置的数据为缺失值。

可以通过 **data[data.isnull().values]** 查看具体的值。

❖ 缺失值处理

● 缺失值检查:

```
data={"name":["Alice","Jack","John","Mary" ],
      "age":[40,10,None,20],
      "weight":[65,30,50,None]}
df=pd.DataFrame(data)
print(df)
#返回一个DataFrame类型数据, 无法通过data[]索引取值
print(df.isnull())
#返回一个array数组, 可以通过data[]索引取值
print(df.isnull().values)
#重新查看有空值的数据
df[df.isnull().values]
```

	name	age	weight
0	Alice	40.0	65.0
1	Jack	10.0	30.0
2	John	NaN	50.0
3	Mary	20.0	NaN

	name	age	weight
0	False	False	False
1	False	False	False
2	False	True	False
3	False	False	True

```
[[False False False]
 [False False False]
 [False  True False]
 [False False  True]]
```

	name	age	weight
2	John	NaN	50.0
3	Mary	20.0	NaN

❖ 缺失值处理

● 缺失值检查:

mask_data.isnull()#查看mask_data的缺失值

```
# 导入 pandas 库并将其简化为 pd
import pandas as pd
# 读取口罩厂数据, 文件路径为根目录
mask_data = pd.read_csv('mask_data.csv')
# 查看mask_data的缺失值
mask_data.isnull()
```

❖ 缺失值处理

● 缺失值检查:

`mask_data.isnull()` #查看mask_data的缺失值

	订单编号	日期	省	订单量	单价	销售额
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	False	False	False	False	False
4	False	False	False	False	False	False
...	...	...	...	...	...	...
101937	False	True	True	True	True	True
101938	False	True	True	False	False	False
101939	False	True	True	False	False	False
101940	False	True	True	False	True	False
101941	False	True	True	False	True	False

❖ 缺失值处理

● 删除缺失值：

data.dropna(axis, how)删除缺失值

axis参数可以传入0或1，0为行，1为列，默认值为0。用于确定是否删除包含缺失值的行或列。

how参数可以传入'any'或'all'，默认为'any'。

'any': 如果存在任何空值，则删除该行或列。

'all': 如果所有值均为空值，则删除该行或列。

❖ 缺失值处理

● 删除缺失值：

df.dropna() 代码返回的是一个删掉所有缺失数据的 **DataFrame** 对象，**不会修改原 DataFrame 对象的数据**，因此**需要重新赋值**给变量来**保存结果**。

删除所有缺失值后重新赋值给变量 mask_data

mask_data = mask_data.dropna()

❖ 缺失值处理

● 删除缺失值：

df.dropna()#将DataFrame对象中包含缺失值的每一行全部删掉。

```
# 导入 pandas 库并将其简化为 pd
import pandas as pd
# 读取口罩厂数据，文件路径为根目录
mask_data = pd.read_csv('mask_data.csv')
# 将DataFrame对象中包含缺失值的每一行全部删掉
mask_data = mask_data.dropna()
# 查看mask_data的缺失值
mask_data.isnull()
```

❖ 缺失值处理

● 删除缺失值:

df.dropna()#将DataFrame对象中包含缺失值的每一行全部删掉。

	订单编号	日期	省	订单量	单价	销售额
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	False	False	False	False	False
4	False	False	False	False	False	False
...	...	...	...	...	...	...
100951	False	False	False	False	False	False
100952	False	False	False	False	False	False
100953	False	False	False	False	False	False
100954	False	False	False	False	False	False
100955	False	False	False	False	False	False

100956 rows × 6 columns

❖ 缺失值处理

● 删除缺失值:

df.dropna()#将DataFrame对象中包含缺失值的每一行全部删掉。

101937	6339-43f4-a12b-7b57d1ce3a48	NaN	NaN	NaN	NaN	NaN
101938	5e43-41c2-8696-fd5909cc98a4	NaN	NaN	999999999.0	999999999.0	999999999.0
101939	824c-4f2b-af43-038938d2db46	NaN	NaN	999999999.0	999999999.0	999999999.0
101940	1cf0-483a-ab11-0cebc6406c78	NaN	NaN	0.0	NaN	0.0
101941	8318-43df-9d34-0656d9eb759a	NaN	NaN	0.0	NaN	0.0

之前



清除完缺失值

100951	8cad-41ee-85b7-c59b85f0ebe6	2020-06-30	湖北	10.0	30.0	300.0
100952	4ea4-480a-a070-295408955363	2020-06-30	湖北	6.0	30.0	180.0
100953	70f3-4461-9b5f-7f53a55c51b6	2020-06-30	湖北	10.0	30.0	300.0
100954	9cc8-4542-8fed-3145e1942504	2020-06-30	湖北	10.0	30.0	300.0
100955	f202-45df-96e2-ca4e4c389624	2020-06-30	湖北	5.0	30.0	150.0

之后

❖ 缺失值处理

● 删除缺失值：

subset参数，它可以在 **dropna()** 方法中指定删除一列或多列数据中含有缺失值的行。

删除'ID'一列数据缺失的行

#将指定的列名写进中括号[]中，再赋值给subset参数，即可限定 dropna() 方法的删除范围

test_data = test_data.dropna(subset=['ID'])

❖ 缺失值处理

● 填补缺失值：

缺失值所在的特征为数值型时，通常利用其均值、中位数和众数等描述其集中趋势的统计量来填充；缺失值所在特征为类别型数据时，则选择众数来填充。

❖ 缺失值处理

● 填补缺失值：

DataFrame.fillna(value=None, method=None, axis=None, inplace=False, limit=None)填补缺失值

value 参数可以传入变量、字典、**Series**、**DataFrame**，用于修改填补缺失值。可以通过字典的形式，针对每一列缺失值进行填补。

inplace参数可以传入**True**或**False**，确认是否在原数据操作。

❖ 缺失值处理

- 填补缺失值： `df.fillna()` # 填补缺失值。

```
df=pd.DataFrame([[None, 3, None, None], [2, 4, None, 3],  
                 [None, None, None, None], [0, 1, 5, 4]],  
                columns=['A', 'B', 'C', 'D'])  
  
print(df)  
#空值用0填充  
print(df.fillna(0, inplace=False))  
#空值用字典填充  
print(df.fillna({'A':0, 'B':1, 'C':2, 'D':3}, inplace=False))
```

	A	B	C	D
0	NaN	3.0	NaN	NaN
1	2.0	4.0	NaN	3.0
2	NaN	NaN	NaN	NaN
3	0.0	1.0	5.0	4.0

	A	B	C	D
0	0.0	3.0	0.0	0.0
1	2.0	4.0	0.0	3.0
2	0.0	0.0	0.0	0.0
3	0.0	1.0	5.0	4.0

	A	B	C	D
0	0.0	3.0	2.0	3.0
1	2.0	4.0	2.0	3.0
2	0.0	1.0	2.0	3.0
3	0.0	1.0	5.0	4.0

❖ 缺失值处理

● 填补缺失值:

```
➤ #通过字典形式填充缺少值
df=pd.DataFrame(np.random.randn(5,3))
df.loc[:3,1]=None
df.loc[:2,2]=None
print(df)
df.fillna({1:0.88,2:0.66})
```

	0	1	2
0	-0.085723	NaN	NaN
1	0.604436	NaN	NaN
2	0.523148	NaN	NaN
3	1.165171	NaN	0.773569
4	0.977133	1.013956	0.686822

df:

	0	1	2
0	-0.085723	0.880000	0.660000
1	0.604436	0.880000	0.660000
2	0.523148	0.880000	0.660000
3	1.165171	0.880000	0.773569
4	0.977133	1.013956	0.686822

● 数据预处理



❖ 缺失值处理

● 填补缺失值:

```
▶ #用Series的均值填充缺失值  
data=pd.Series([1, None, 3.5, None, 7])  
data1=data.fillna(data.mean())  
print(data)  
print(data1)
```

```
0    1.0  
1    NaN  
2    3.5  
3    NaN  
4    7.0  
dtype: float64  
0    1.000000  
1    3.833333  
2    3.500000  
3    3.833333  
4    7.000000  
dtype: float64
```

❖ 缺失值处理

● 填补缺失值:

```
▶ #DataFrame中用均值填充
df=pd.DataFrame(np.random.randn(4,3))
df.iloc[2:,1]=None
df.iloc[3:,2]=None
print(df)
df[1]=df[1].fillna(df[1].mean())
print(df)
```

	0	1	2
0	1.273512	-1.050069	-0.345632
1	-1.828834	2.615134	-0.282838
2	0.723698	NaN	0.189533
3	0.352419	NaN	NaN

	0	1	2
0	1.273512	-1.050069	-0.345632
1	-1.828834	2.615134	-0.282838
2	0.723698	0.782532	0.189533
3	0.352419	0.782532	NaN

❖ 缺失值处理

● 填补缺失值：

df.fillna(method='ffill') 用前一个数据值替代NaN

填补缺失值。

❖ 缺失值处理

● 填补缺失值：

#DataFrame中用前向值填充缺失值

```
df=pd.DataFrame(np.random.randn(6,3))
```

```
df.iloc[2:,1]=None
```

```
df.iloc[4:,2]=None
```

```
print(df)
```

```
df.fillna(method='ffill')
```

	0	1	2
0	-0.554021	-0.321301	-0.771191
1	-0.427392	-0.512386	-1.297963
2	-0.876656	NaN	-0.052016
3	-1.601495	NaN	-0.489785
4	1.540123	NaN	NaN
5	0.361237	NaN	NaN

	0	1	2
0	-0.554021	-0.321301	-0.771191
1	-0.427392	-0.512386	-1.297963
2	-0.876656	-0.512386	-0.052016
3	-1.601495	-0.512386	-0.489785
4	1.540123	-0.512386	-0.489785
5	0.361237	-0.512386	-0.489785

❖ 缺失值处理

● 填补缺失值：

`df.fillna(method='bfill')` 用后一个数据值替代NaN，与pad相反，bfill表示用后一个数据代替NaN，可以用**limit**限制每列可以替代NaN的数目。

❖ 缺失值处理

- **df.fillna(method='bfill')** 用后一个数据值替代NaN。

```
▶ import numpy as np
import pandas as pd
df=pd.DataFrame(np.random.randn(6,3))
df.iloc[2:4,1]=None
df.iloc[3:4,2]=None
print(df)
df.fillna(method='bfill')
```

	0	1	2
0	-1.154883	0.295889	0.051049
1	-0.797713	-0.792751	-0.176408
2	-0.047639	NaN	0.386015
3	-1.568716	NaN	NaN
4	-1.209246	0.046185	0.625937
5	1.236301	2.094383	0.476643

21]:

	0	1	2
0	-1.154883	0.295889	0.051049
1	-0.797713	-0.792751	-0.176408
2	-0.047639	0.046185	0.386015
3	-1.568716	0.046185	0.625937
4	-1.209246	0.046185	0.625937
5	1.236301	2.094383	0.476643

❖ 缺失值处理

- `df.fillna(df.mean())` 用平均数或者其他描述性统计量来代替 NaN。先计算出该列的平均数或其他描述性统计量，再将该列的缺失值用计算出的值进行填充。

❖ 缺失值处理

```
#用均值来填充空值
data={ "name":["Alice","Jack","John","Mary" ],
        "math":[76,85, None,87],
        "physical":[85,56,95,75],
        "Chinese":[78, None,85,58]}
df=pd.DataFrame(data)
print(df)
df['math'].fillna(df['math'].mean(),inplace=True)
df['Chinese'].fillna(df['Chinese'].mean(),inplace=True)
print(df)
```

	name	math	physical	Chinese
0	Alice	76.0	85	78.0
1	Jack	85.0	56	NaN
2	John	NaN	95	85.0
3	Mary	87.0	75	58.0

	name	math	physical	Chinese
0	Alice	76.000000	85	78.000000
1	Jack	85.000000	56	73.666667
2	John	82.666667	95	85.000000
3	Mary	87.000000	75	58.000000

❖ 缺失值处理

● `df.fillna(df.mean()['math': 'Chinese'])` 可以选择列进行缺失值的处理。

```
data={"name":["Alice","Jack","John","Mary"],
      "math":[76,85,None,87],
      "physical":[85,56,95,75],
      "Chinese":[78,None,85,58]}
df=pd.DataFrame(data)
print(df)
df.fillna(df.mean()['math':'Chinese'])
```

	name	math	physical	Chinese
0	Alice	76.0	85	78.0
1	Jack	85.0	56	NaN
2	John	NaN	95	85.0
3	Mary	87.0	75	58.0

]:

	name	math	physical	Chinese
0	Alice	76.000000	85	78.000000
1	Jack	85.000000	56	73.666667
2	John	82.666667	95	85.000000
3	Mary	87.000000	75	58.000000

❖ 缺失值处理

● 填补缺失值：

在填补缺失值的时候，通常使用平均数、中位数、众数等等这类可以代表一组数据典型情况的数据进行填补。一般来说，用平均数进行填补较为普遍。但是平均数要求对所有数据进行加总计算，所以只适用于数值型数据。当缺失的数据是分类数据或顺序数据时，就无法使用这种填补方式。

❖ 缺失值处理

● 填补缺失值：

求平均数： **data.mean()**

求中位数： **data.median()**

求众数： **data.mode()**

求最大值： **data.max()**

求最小值： **data.min()**

求和： **data.sum()**

求标准差： **data.std()**

❖ 字符型数据中的空格处理

- **strip()**: 清除字符型数据左右(首尾)指定的字符, 默认为空格, 中间的不清除。

```
#strip()清除字符型数据左右(首尾)指定的空格
data={  "name":["Alice","Jack","John","Mary" ],
        "IP":[" 221.205.98.55 ", " 183.184.226.205",
        " 221.205.98.55 ", " 222.31.51.200      "],
        "年龄":[25,44,46,34],
        "性别":["女",'男','男','女' ]}
df=pd.DataFrame(data)
print(df)
#因为IP是一个对象, 所以先转为str
newDF=df['IP'].str.strip()
newDF
```

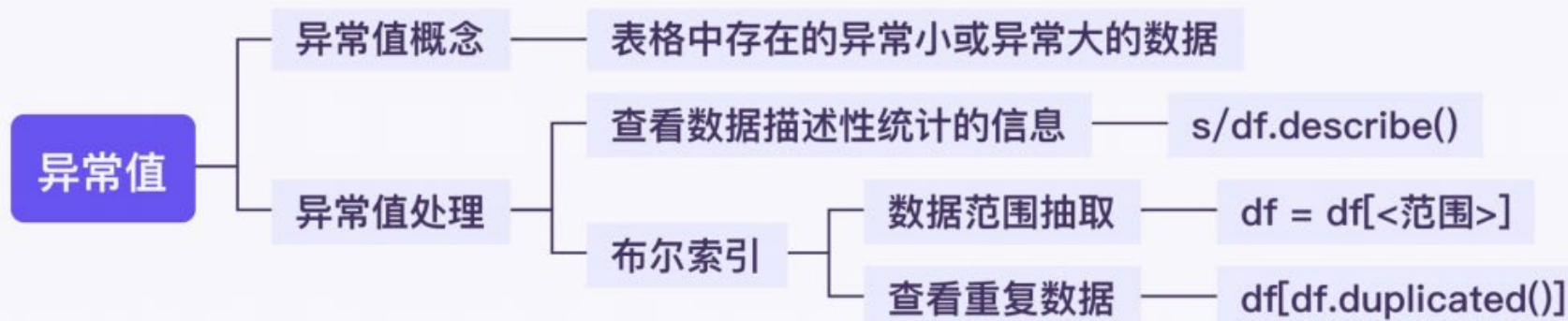
❖ 空格处理

	name	IP	年龄	性别
0	Alice	221.205.98.55	25	女
1	Jack	183.184.226.205	44	男
2	John	221.205.98.55	46	男
3	Mary	222.31.51.200	34	女

	name	IP	年龄	性别
0	Alice	221.205.98.55	25	女
1	Jack	183.184.226.205	44	男
2	John	221.205.98.55	46	男
3	Mary	222.31.51.200	34	女

❖ 处理异常值

- 有时数据中有一个或多个**异常大**或**异常小**的数值，**超出了**这份数据实际的**限定范围**，这样的数值被称为**异常值**。



❖ 处理异常值

● describe() 检查异常值

df.describe() 方法可以查看Series对象或者DataFrame对象描述性统计信息。

```
mask_data.describe()
```

	订单量	单价	销售额
count	1.009560e+05	1.009560e+05	1.009560e+05
mean	9.509401e+05	9.510114e+05	9.549671e+05
std	3.082230e+07	3.082230e+07	3.082218e+07
min	0.000000e+00	3.000000e+01	0.000000e+00
25%	9.000000e+00	5.000000e+01	5.500000e+02
50%	2.000000e+01	5.000000e+01	1.700000e+03
75%	4.700000e+01	1.500000e+02	4.000000e+03
max	1.000000e+09	1.000000e+09	1.000000e+09

❖ 处理异常值

● describe() 检查异常值

	订单量	单价	销售额
count	1.009560e+05	1.009560e+05	1.009560e+05
mean	9.509401e+05	9.510114e+05	9.549671e+05
std	3.082230e+07	3.082230e+07	3.082218e+07
min	0.000000e+00	3.000000e+01	0.000000e+00
25%	9.000000e+00	5.000000e+01	5.500000e+02
50%	2.000000e+01	5.000000e+01	1.700000e+03
75%	4.700000e+01	1.500000e+02	4.000000e+03
max	1.000000e+09	1.000000e+09	1.000000e+09

❖ 处理异常值

● 用 **describe()** 检查异常值，发现如下异常情况：

1) 订单量的最小值几乎是 0

在实际业务中，不可能出现订单量为 0 的情况，所以订单量等于 0 的数据属于异常值。

2) 订单量的最大值几乎是 10 的九次方

10 个亿的订单量我司口罩厂的产能明显无法跟上，所以这种极大的数据也属于异常值。

3) 单价的最大值也几乎是 10 的九次方

公司每盒口罩的单价最高 200 元，大于 200 的数据都属于异常值。

❖ 布尔索引

- 在pandas库中，有一种筛选数据的方法叫做布尔索引，使用`df[ ]`通过表达式的形式来提取一定范围的数据。
- 使用 `df['列索引']` 来提取某一列的信息。在 pandas 库中，还可以在 `df[]` 中通过表达式的形式来提取一定范围的数据。比如想选择 `mask_data` 数据中，单价一列小于等于 200 的数据，就可以设置条件表达式 `mask_data['单价'] <= 200`。把它加入括号中索引：`mask_data[mask_data['单价'] <= 200]`，就可以选取 `mask_data` 数据中单价这一列小于等于 200 的数据。

❖ 布尔索引

筛选单价小于等于 200 的数据

mask_data = mask_data[mask_data['单价'] <= 200]

```
# 筛选单价小于等于 200 的数据  
mask_data = mask_data[mask_data['单价'] <= 200]  
# 查看mask_data的描述性统计信息  
mask_data.describe()
```

	订单量	单价	销售额
count	100860.000000	100860.000000	100860.000000
mean	30.786308	102.180349	4061.675193
std	27.640552	63.541341	5278.716290
min	0.000000	30.000000	0.000000
25%	9.000000	50.000000	550.000000
50%	20.000000	50.000000	1700.000000
75%	47.000000	150.000000	4000.000000
max	100.000000	200.000000	20000.000000

❖ 布尔索引

筛选mask_data中订单量大于0的数据

`mask_data = mask_data[mask_data['订单量'] > 0]`

```
# 筛选订单量大于 0 的数据  
mask_data = mask_data[mask_data['订单量'] > 0]  
# 查看 mask_data 的描述性统计信息  
mask_data.describe()
```

	订单量	单价	销售额
count	99926.000000	99926.000000	99926.000000
mean	31.074065	102.783960	4099.639333
std	27.607959	63.493715	5288.634780
min	1.000000	30.000000	30.000000
25%	9.000000	50.000000	560.000000
50%	20.000000	100.000000	1750.000000
75%	47.000000	150.000000	4000.000000
max	100.000000	200.000000	20000.000000

❖ 处理数据格式错误

- 数据格式错误是数据清洗中最常见的问题类型之一，指数据在结构、模式或表示方式上不符合预期标准的情况。
- 数据格式错误是指数据在语法和结构上违反预定规则的问题，不同于数据值本身的错误（如异常值）。
- **dataframe** 数据清洗纠正格式错误：例如，将字符串 "42.5%" 转换为数值 0.425；将日期 "2023/13/01" 修正为 "2024/01/01" 或标记为缺失值。

❖ 处理数据格式错误

处理数据格式错误分为整列格式错误和其中个别数据格式错误两种情况：

- 第一种情况：当某列数据的类型不符合要求时，可以使用 `Series.astype()` 函数进行该列数据类型的强制转换。
- 第二种情况：有时在使用 `Series.astype()` 函数时会遇到程序报错，证明该列的数据类型不统一，这时就需要先进行错误定位，然后将其修改为正确的数据即可。

❖ 处理数据格式错误

● 第一种情况常见的时间格式的处理，可以使用 `pd.to_datetime(arg, format, errors)` 进行时间格式化。

arg 参数可以传入需要处理的内容，可以是 `int`、`float`、`str`、`datetime`、`list`、`tuple` 等。

format 参数传入的值需要对应待处理数据的格式，如 `'20201201'` 对应 `'%Y%m%d'`、`'11042021'` 对应 `'%d%m%Y'`。

errors 参数可以传入 `{'ignore', 'raise', 'coerce'}`，默认为 `'raise'`。

如果为 `'raise'`，当无法解析时程序会报错，如 `20200231`，2月没有31号，程序会报错；

如果为 `'coerce'`，当无法解析时，将内容设置为空值；

如果为 `'ignore'`，当无法解析时，忽视该数据。

❖ 处理数据格式错误

- 第一种情况：当某列数据的类型不符合要求时，可以使用 `Series.astype()` 函数进行该列数据类型的强制转换。

注意：在使用 `pd.to_datetime()` 函数转换入职日期时，一定要先清除日期字符串两边的空格，再进行转换，否则转换出来有可能是 `NaT` 值。



```
import pandas as pd
import numpy as np
# 创建包含数据格式错误的DataFrame
data = {
    '年龄': ['25', '30', '35', '40', '50'],
    '工资': ['5000.50', '7500', '6000', '9000.75', '12000'],
    # 假设原始数据中可能存在不规范的空格
    '入职日期': [' 2023-01-15', '2022-06-20 ', '2023-12-01', '2024-02-05', '2024-03-10'],
    '部门编号': ['001', '002', '003', '004', '005'],
    '绩效评分': [86, 79, 95, 87.5, 93]
}
df = pd.DataFrame(data)
# --- 开始修正 ---
# 1. 转换年龄为整数
df['年龄'] = df['年龄'].astype('int')
# 2. 转换工资为浮点数
df['工资'] = df['工资'].astype('float')
# 3. 【关键步骤】先清除日期字符串两边的空格，再进行转换
df['入职日期'] = pd.to_datetime(df['入职日期'].str.strip(), format='%Y-%m-%d')
# --- 修正结束 ---
print(df)
print("\n转换后的各列数据类型:")
print(df.dtypes)
```

	年龄	工资	入职日期	部门编号	绩效评分
0	25	5000.50	2023-01-15	001	86.0
1	30	7500.00	2022-06-20	002	79.0
2	35	6000.00	2023-12-01	003	95.0
3	40	9000.75	2024-02-05	004	87.5
4	50	12000.00	2024-03-10	005	93.0

转换后的各列数据类型:

```
年龄          int32
工资          float64
入职日期      datetime64[ns]
部门编号      object
绩效评分      float64
dtype: object
```

❖ 处理数据格式错误

- 第二种情况：当使用`data.astype()`遇到报错时，证明该列的数据类型不统一，这时就需要先进行错误定位，然后将其修改为正确的数据即可。

❖ 处理数据格式错误（定位错误数据）

```
import pandas as pd
import numpy as np
# 创建包含数据格式错误的DataFrame
data = {
    '年龄': ['25', '30', '三十五', '40', 'abc'], # 字符串包含中文和字母
    '工资': [5000.50, 7500, 6000, 9000.75, 12000],
    '部门编号': ['001', '002', '003', '004', '005'],
    '绩效评分': [86, 79, 95, 87.5, 93]
}
df = pd.DataFrame(data)
# 定位并修正DataFrame中的格式错误
def locate_and_fix_errors(df):
    df_fixed = df.copy()
    error_report = {}
    # 检查年龄列（应为数值）
    print("=== 检查年龄列 ===")
    age_errors = pd.to_numeric(df_fixed['年龄'], errors='coerce').isna()
    if age_errors.any():
        error_indices = df_fixed[age_errors].index.tolist()
        error_report['年龄'] = error_indices
        print(f"年龄列错误行索引: {error_indices}")
        print("错误数据:")
        print(df_fixed.loc[error_indices, ['姓名', '年龄']])
    locate_and_fix_errors(df)
```

```
=== 检查年龄列 ===
年龄列错误行索引: [2, 4]
错误数据:
```

❖ 处理数据格式错误（修改错误数据）

```
df['年龄'].iloc[2]=52  
df['年龄'].iloc[4]=60  
print(df['年龄'])
```

```
0    25  
1    30  
2    52  
3    40  
4    60  
Name: 年龄, dtype: object
```

❖ 数据整理

- 数据清洗完毕后并不意味着这份数据可以直接拿去分析。
- 在打扫房间时不光要把垃圾倒掉，还需要将凌乱的物品摆放整齐。
- 数据处理也是一样，不光要把数据清洗干净，还需要整理数据将数据变得规整，这样才有利于后续的数据分析。

❖ 数据整理

- **数据整理**指的是在数据分析前对所需字段进行**数据排序**、**数据转换**、**数据抽取**、**数据合并**、**数据计算**等准备操作。



❖ 数据整理

在开始整理数据之前，我们先观察一下清洗好的数据，看看有哪些字段可以被整理。

```
# 查看清洗好的数据  
mask_data
```

	订单编号	日期	省	订单量	单价	销售额
2	a56d-4415-ad6e-020cdb154c35	2020-01-01	湖北	1.0	30.0	30.0
6	953f-4b46-a8a2-0eb0e39818a8	2020-01-01	湖北	1.0	30.0	30.0
11	87af-48e5-8bed-c5dcf9ecc172	2020-01-01	湖北	1.0	30.0	30.0
12	535a-4eca-8fa0-9cc54c66e11d	2020-01-01	湖南	1.0	30.0	30.0
13	d707-4d5c-ac10-59d3fd3dc0bb	2020-01-01	广东	1.0	30.0	30.0
...	...	...	...	...	...	...
100951	8cad-41ee-85b7-c59b85f0ebe6	2020-06-30	湖北	10.0	30.0	300.0
100952	4ea4-480a-a070-295408955363	2020-06-30	湖北	6.0	30.0	180.0
100953	70f3-4461-9b5f-7f53a55c51b6	2020-06-30	湖北	10.0	30.0	300.0
100954	9cc8-4542-8fed-3145e1942504	2020-06-30	湖北	10.0	30.0	300.0
100955	f202-45df-96e2-ca4e4c389624	2020-06-30	湖北	5.0	30.0	150.0

99926 rows × 6 columns

❖ 数据整理

从结果中可以发现，日期这一列数据的格式是年-月-日，也就是说，我们的统计数据是精确到日的。

	订单编号	日期	省	订单量	单价	销售额
2	a56d-4415-ad6e-020cdb154c35	2020-01-01	湖北	1.0	30.0	30.0
6	953f-4b46-a8a2-0eb0e39818a8	2020-01-01	湖北	1.0	30.0	30.0
11	87af-48e5-8bed-c5dcf9ecc172	2020-01-01	湖北	1.0	30.0	30.0
12	535a-4eca-8fa0-9cc54c66e11d	2020-01-01	湖南	1.0	30.0	30.0
13	d707-4d5c-ac10-59d3fd3dc0bb	2020-01-01	广东	1.0	30.0	30.0
...	...	...	...	...	...	...
100951	8cad-41ee-85b7-c59b85f0ebe6	2020-06-30	湖北	10.0	30.0	300.0
100952	4ea4-480a-a070-295408955363	2020-06-30	湖北	6.0	30.0	180.0
100953	70f3-4461-9b5f-7f53a55c51b6	2020-06-30	湖北	10.0	30.0	300.0
100954	9cc8-4542-8fed-3145e1942504	2020-06-30	湖北	10.0	30.0	300.0
100955	f202-45df-96e2-ca4e4c389624	2020-06-30	湖北	5.0	30.0	150.0

❖ 数据整理

假设要先分析研究各个月的利润变化情况，所以需要将日期的统计数据由年-月-日调整为月，然后作为新的月份字段保存到数据中。

	订单编号	日期	省	订单量	单价	销售额			订单编号	日期	省	订单量	单价	销售额	月份	
0	a56d-4415-ad6e-020cdb154c35	2020-01-01	湖北	1.0	30.0	30.0	0	1	0	a56d-4415-ad6e-020cdb154c35	2020-01-01	湖北	1.0	30.0	30.0	1
1	953f-4b46-a8a2-0eb0e39818a8	2020-01-01	湖北	1.0	30.0	30.0	1	1	1	953f-4b46-a8a2-0eb0e39818a8	2020-01-01	湖北	1.0	30.0	30.0	1
2	87af-48e5-8bed-c5dcf9ecc172	2020-01-01	湖北	1.0	30.0	30.0	2	1	2	87af-48e5-8bed-c5dcf9ecc172	2020-01-01	湖北	1.0	30.0	30.0	1
3	535a-4eca-8fa0-9cc54c66e11d	2020-01-01	湖南	1.0	30.0	30.0	3	1	3	535a-4eca-8fa0-9cc54c66e11d	2020-01-01	湖南	1.0	30.0	30.0	1
4	d707-4d5c-ac10-59d3fd3dc0bb	2020-01-01	广东	1.0	30.0	30.0	4	1	4	d707-4d5c-ac10-59d3fd3dc0bb	2020-01-01	广东	1.0	30.0	30.0	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
99480	8cad-41ee-85b7-c59b85f0ebe6	2020-06-30	湖北	10.0	30.0	300.0	99480	6	99480	8cad-41ee-85b7-c59b85f0ebe6	2020-06-30	湖北	10.0	30.0	300.0	6
99481	4ea4-480a-a070-295408955363	2020-06-30	湖北	6.0	30.0	180.0	99481	6	99481	4ea4-480a-a070-295408955363	2020-06-30	湖北	6.0	30.0	180.0	6
99482	70f3-4461-9b5f-7f53a55c51b6	2020-06-30	湖北	10.0	30.0	300.0	99482	6	99482	70f3-4461-9b5f-7f53a55c51b6	2020-06-30	湖北	10.0	30.0	300.0	6
99483	9cc8-4542-8fed-3145e1942504	2020-06-30	湖北	10.0	30.0	300.0	99483	6	99483	9cc8-4542-8fed-3145e1942504	2020-06-30	湖北	10.0	30.0	300.0	6
99484	f202-45df-96e2-ca4e4c389624	2020-06-30	湖北	5.0	30.0	150.0	99484	6	99484	f202-45df-96e2-ca4e4c389624	2020-06-30	湖北	5.0	30.0	150.0	6

❖ 数据整理

但是mask_data 中的日期列数据并不是datetime类型。

```
mask_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 99926 entries, 2 to 100955
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   订单编号    99926 non-null  object
1   日期        99926 non-null  object
2   省          99926 non-null  object
3   订单量      99926 non-null  float64
4   单价        99926 non-null  float64
5   销售额      99926 non-null  float64
dtypes: float64(3), object(3)
memory usage: 5.3+ MB
```

❖ 数据整理

- Python 中有一种专门储存时间日期的数据类型 —— `datetime`，可以直接从中提取想要的时间单位。
- 需要将日期这一列的数据类型转换成 `datetime` 类型，才能从中提取月份信息。
- 因此可以将这次的数据整理分成了三个步骤，它们分别是：转换日期数据、提取月份信息以及添加新列。

❖ 数据整理（转换日期数据）

pd.to_datetime(arg, format) 转换日期数据

使用 `pd.to_datetime(arg, format)` 来将 `DataFrame` 对象或者 `Series` 对象的数据类型转换成 `datetime` 类型。

`arg` 参数为要转换的数据，它可以是 `DataFrame` 对象或 `Series` 对象。

`format` 参数为 `datetime` 类型的日期格式，比如说这份数据，它是以年-月-日的形式出现的，

那么它对应的 `format` 就是 `'%Y-%m-%d'`（year-month-day 的缩写）。

❖ 数据整理（转换日期数据）

```
# 转换日期数据，并设置对应的日期格式
```

```
date_data = pd.to_datetime(mask_data['日期'], format = '%Y-%m-%d')
```

```
# 查看 date_data
```

```
date_data
```

```
2          2020-01-01
```

```
6          2020-01-01
```

```
11         2020-01-01
```

```
12         2020-01-01
```

```
13         2020-01-01
```

```
...
```

```
100951     2020-06-30
```

```
100952     2020-06-30
```

```
100953     2020-06-30
```

```
100954     2020-06-30
```

```
100955     2020-06-30
```

```
Name: 日期, Length: 99926, dtype: datetime64[ns]
```

❖ 数据整理（提取月份信息）

Series.dt.month 提取月份信息

Series.dt 可以把datetime类型的数据转成便于我们提取对应的日期和时间数据的对象。

日期数据属性

s.dt.year

返回年份信息

s.dt.month

返回月份信息

s.dt.day

返回某日信息

❖ 数据整理（提取月份信息）

```
# 转换日期数据，并设置对应的日期格式
date_data = pd.to_datetime(mask_data['日期'], format = '%Y-%m-%d')
# 提取日期数据中的月份信息
month_data = date_data.dt.month
# 查看 month_data
month_data
```

```
2          1
6          1
11         1
12         1
13         1
..
100951     6
100952     6
100953     6
100954     6
100955     6
Name: 日期, Length: 99926, dtype: int64
```

❖ 数据整理（添加新列）

添加新列

使用`df['colname'] = Series`的方式来为原数据添加新的一列。

`df`是`DataFrame`对象的简称。

`colname`指的是要添加的新列的列名，在此案例中，`colname`就是月份。

`Series`指的是要传入的`Series`对象，在此案例中，`s`就是上一页得到的`month_data`。

❖ 数据整理（添加新列）

```
# 将月份数据添加到原数据中  
mask_data['月份'] = month_data  
# 查看原数据  
mask_data
```

	订单编号	日期	省	订单量	单价	销售额	月份
2	a56d-4415-ad6e-020cdb154c35	2020-01-01	湖北	1.0	30.0	30.0	1
6	953f-4b46-a8a2-0eb0e39818a8	2020-01-01	湖北	1.0	30.0	30.0	1
11	87af-48e5-8bed-c5dcf9ecc172	2020-01-01	湖北	1.0	30.0	30.0	1
12	535a-4eca-8fa0-9cc54c66e11d	2020-01-01	湖南	1.0	30.0	30.0	1
13	d707-4d5c-ac10-59d3fd3dc0bb	2020-01-01	广东	1.0	30.0	30.0	1
...	...	...	...	...	...	...	...
100951	8cad-41ee-85b7-c59b85f0ebe6	2020-06-30	湖北	10.0	30.0	300.0	6
100952	4ea4-480a-a070-295408955363	2020-06-30	湖北	6.0	30.0	180.0	6
100953	70f3-4461-9b5f-7f53a55c51b6	2020-06-30	湖北	10.0	30.0	300.0	6
100954	9cc8-4542-8fed-3145e1942504	2020-06-30	湖北	10.0	30.0	300.0	6
100955	f202-45df-96e2-ca4e4c389624	2020-06-30	湖北	5.0	30.0	150.0	6

99926 rows × 7 columns

❖ 常用的数据整理函数

- 数据整理有时还需要对所需字段进行数据合并，下面介绍二种常见的数据合并函数：

1. merge数据合并

merge函数是通过一个或多个键将两个**DataFrame**按行合并起来，与SQL中的 **join** 用法类似，**Pandas**中的数据合并**merge()**函数格式如下：

```
merge(left, right, how='inner', on=None, left_on=None, right_on=None, left_index=False, right_index=False, sort=False, suffixes=('_x', '_y'), copy=True, indicator=False, validate=None)
```

❖ 常用的数据整理函数

1. merge数据合并

merge方法主要参数及说明

参数	使用说明
left	参与合并的左侧 DataFrame
right	参与合并的右侧 DataFrame
how	连接方法:inner, left, right, outer
on	用于连接的列名
left_on	左侧 DataFrame 中用于连接键的列
right_on	右侧 DataFrame 中用于连接键的列
left_index	左侧 DataFrame 中行索引作为连接键
right_index	右侧 DataFrame 中行索引作为连接键
sort	合并后会对数据排序, 默认为 True
suffixes	修改重复名

❖ 常用的数据整理函数

1. merge数据合并

#merge数据合并

```
price = pd.DataFrame({'fruit': ['apple', 'grape', 'orange', 'orange'],  
                      'price': [8, 7, 9, 11]})  
amount = pd.DataFrame({'fruit': ['apple', 'grape', 'orange'],  
                       'amount': [5, 11, 8]})  
display(price, amount, pd.merge(price, amount))
```

	fruit	price
0	apple	8
1	grape	7
2	orange	9
3	orange	11

	fruit	amount
0	apple	5
1	grape	11
2	orange	8

	fruit	price	amount
0	apple	8	5
1	grape	7	11
2	orange	9	8
3	orange	11	8

❖ 常用的数据整理函数

2. concat数据连接

如果要合并的**DataFrame**之间没有连接键，就无法使用**merge**方法。**pandas**中的**concat**方法可以实现，默认情况下会按行的方向堆叠数据。如果在列向上连接设置**axes = 1**即可。

```
#concat数据连接
s1=pd.Series([0,1],index=['a','b'])
s2=pd.Series([2,3,4],index=['a','d','e'])
s3=pd.Series([5,6],index=['f','g'])
print(pd.concat([s1,s2,s3])) #Series行合并
```

```
a    0
b    1
a    2
d    3
e    4
f    5
g    6
dtype: int64
```

❖ 常用的数据整理函数

2. concat数据连接

#concat数据连接

```
data1=pd.DataFrame(np.arange(6).reshape(2,3),columns=list('abc'))  
data2=pd.DataFrame(np.arange(20,26).reshape(2,3),columns=list('ayz'))  
data=pd.concat([data1,data2],axis=0)  
display(data1,data2,data)
```

	a	b	c
0	0	1	2
1	3	4	5

	a	y	z
0	20	21	22
1	23	24	25

	a	b	c	y	z
0	0	1.0	2.0	NaN	NaN
1	3	4.0	5.0	NaN	NaN
0	20	NaN	NaN	21.0	22.0
1	23	NaN	NaN	24.0	25.0

❖ 数据整理（数据写入）

- 使用 `df.to_csv()` 方法，将清洗好的数据写入 `csv` 文件中。
- 使用 `df.to_csv()` 方法的 `index` 参数来限制 `DataFrame` 对象的行索引是否写入到 `csv` 文件。它的默认值为 `True`，会让写入的 `csv` 文件包含一列行索引。如果把这个参数设置为 `index = False` 就可以取消写入行索引，不用将行索引那一列写入 `csv` 文件。

```
# 保存清洗干净的数据，并取消写入行索引  
mask_data.to_csv('mask_data_clean.csv', index = False)
```



❖ 数据整理（数据写入）

```
mask_data_clean.csv [C:\2021-2022 (2) \bigdata\jupyter code] - Notepad3
文件(F) 编辑(E) 查看(V) 外观(P) 设置(S) 帮助(H)
1 订单编号,日期,省,订单量,单价,销售额,月份
2 a56d-4415-ad6e-020cdb154c35,2020-01-01,湖北,1.0,30.0,30.0,1
3 953f-4b46-a8a2-0eb0e39818a8,2020-01-01,湖北,1.0,30.0,30.0,1
4 87af-48e5-8bed-c5dcf9ecc172,2020-01-01,湖北,1.0,30.0,30.0,1
5 535a-4eca-8fa0-9cc54c66e11d,2020-01-01,湖南,1.0,30.0,30.0,1
6 d707-4d5c-ac10-59d3fd3dc0bb,2020-01-01,广东,1.0,30.0,30.0,1
7 a56d-4415-ad6e-020cdb154c35,2020-01-01,河南,1.0,30.0,30.0,1
8 953f-4b46-a8a2-0eb0e39818a8,2020-01-01,江苏,1.0,30.0,30.0,1
9 70f3-4461-9b5f-7f53a55c51b6,2020-01-01,广东,1.0,30.0,30.0,1
10 a56d-4415-ad6e-020cdb154c35,2020-01-01,其他,1.0,30.0,30.0,1
11 7b1f-49d1-991e-6480ffe0c6c4,2020-01-01,湖北,1.0,30.0,30.0,1
12 2b31-4392-be6e-19af71e40261,2020-01-01,湖北,1.0,30.0,30.0,1
13 d707-4d5c-ac10-59d3fd3dc0bb,2020-01-01,江苏,1.0,30.0,30.0,1
14 70f3-4461-9b5f-7f53a55c51b6,2020-01-01,湖北,1.0,30.0,30.0,1
15 87af-48e5-8bed-c5dcf9ecc172,2020-01-01,江苏,1.0,30.0,30.0,1
16 a56d-4415-ad6e-020cdb154c35,2020-01-01,湖南,1.0,30.0,30.0,1
17 87af-48e5-8bed-c5dcf9ecc172,2020-01-01,湖南,1.0,30.0,30.0,1
18 d707-4d5c-ac10-59d3fd3dc0bb,2020-01-01,河南,1.0,30.0,30.0,1
19 535a-4eca-8fa0-9cc54c66e11d,2020-01-01,江苏,1.0,30.0,30.0,1
20 953f-4b46-a8a2-0eb0e39818a8,2020-01-01,广东,1.0,30.0,30.0,1
21 a56d-4415-ad6e-020cdb154c35,2020-01-01,江苏,1.0,30.0,30.0,1
22 2b31-4392-be6e-19af71e40261,2020-01-02,河南,1.0,30.0,30.0,1
23 953f-4b46-a8a2-0eb0e39818a8,2020-01-02,其他,1.0,30.0,30.0,1
24 535a-4eca-8fa0-9cc54c66e11d,2020-01-02,河南,1.0,30.0,30.0,1
25 d707-4d5c-ac10-59d3fd3dc0bb,2020-01-02,湖南,1.0,30.0,30.0,1
26 535a-4eca-8fa0-9cc54c66e11d,2020-01-02,其他,1.0,30.0,30.0,1
27 a56d-4415-ad6e-020cdb154c35,2020-01-02,其他,1.0,30.0,30.0,1
28 d707-4d5c-ac10-59d3fd3dc0bb,2020-01-02,其他,1.0,30.0,30.0,1
29 535a-4eca-8fa0-9cc54c66e11d,2020-01-02,湖北,1.0,30.0,30.0,1
30 2b31-4392-be6e-19af71e40261,2020-01-02,其他,1.0,30.0,30.0,1
31 87af-48e5-8bed-c5dcf9ecc172,2020-01-02,湖南,1.0,30.0,30.0,1
32 dd85-4865-920a-a6a14f50f97f,2020-01-02,广东,1.0,30.0,30.0,1
33 a56d-4415-ad6e-020cdb154c35,2020-01-02,广东,1.0,30.0,30.0,1
34 70f3-4461-9b5f-7f53a55c51b6,2020-01-02,江苏,1.0,30.0,30.0,1
35 a56d-4415-ad6e-020cdb154c35,2020-01-02,湖北,1.0,30.0,30.0,1
36 d707-4d5c-ac10-59d3fd3dc0bb,2020-01-02,广东,1.0,30.0,30.0,1
```